

HANDS-ON KUBERNETES LAB

Kubernetes Networking: Services, Ingress & Network Policies

A full hands-on lab covering ClusterIP, NodePort, LoadBalancer, ExternalName, Ingress, and NetworkPolicies — built directly against your own 3-node bare-metal cluster.

ClusterIP

NodePort

LoadBalancer

ExternalName

Ingress

NetworkPolicy

control-plane	k8s-master	•	192.168.163.128
worker node 1	k8s-worker1.it24.local	•	192.168.163.129
worker node 2	k8s-worker2.it24.local	•	192.168.163.130

Lab Goal

Kubernetes Services all solve the same problem — giving a stable way to reach a moving set of Pods — but they solve it at **different layers** and for **different audiences**. This lab walks through every Service type on your real cluster, shows exactly when each one is (and isn't) reachable, then layers on **NetworkPolicies** to control traffic between Pods, and finishes with an **Ingress** challenge that ties everything together.

Your Cluster

```
k8sadmin@k8s-master:~$ kubectl get nodes
NAME                STATUS    ROLES          AGE   VERSION
k8s-master          Ready    control-plane  2d3h  v1.35.6
k8s-worker1.it24.local  Ready    <none>         2d3h  v1.35.6
k8s-worker2.it24.local  Ready    <none>         2d3h  v1.35.6
```



No cloud Load Balancer: this is a bare-metal / on-prem cluster, so there is no cloud provider to hand out an external IP automatically. We'll see exactly what that looks like in Part 3, and what a real environment would need instead (a cloud LB, or a bare-metal alternative like MetalLB).

CONTENTS

PART 1 — FOUNDATIONS

01	Why Services exist	Theory
----	--------------------	--------

PART 2 — CLUSTERIP

02	Theory & diagram	
03	Hands-on: deploy & test internal-only access	Task

PART 3 — NODEPORT

04	Theory & diagram	
05	Hands-on: reach the app via all 3 node IPs	Task

PART 4 — LOADBALANCER

06	Theory, pending state & MetalLB	
07	Hands-on: observe <pending>	Task

PART 5 — EXTERNALNAME

08	Theory & hands-on	Task
----	-------------------	------

PART 6 — COMPARE & QUIZ

09	Service types cheat sheet	Quiz
----	---------------------------	------

PART 7 — NETWORKPOLICIES

10	Theory, default-allow & CNI check	
11	Hands-on: default deny-all	Task
12	Hands-on: allow frontend → backend	Task
13	Hands-on: lock down the database tier	Task

PART 8 — INGRESS + FINAL CHALLENGE

14	Ingress recap	
15	Final challenge: full 3-tier app	Challenge
16	Troubleshooting cheat sheet	Reference

Why Services Exist

Pods are **ephemeral** — they get rescheduled, restarted, and replaced constantly, and each new Pod gets a new IP. A **Service** is a stable virtual address in front of a changing set of Pods, matched by a **label selector**. Kubernetes has five Service types; this lab covers the four that describe *how traffic reaches the Service*, plus Ingress which routes *on top of* a Service.

TYPE	REACHABLE FROM	TYPICAL USE
ClusterIP	Inside the cluster only	Service-to-service traffic (default)
NodePort	Any node IP, on a high port	Simple external access, dev/test, on-prem
LoadBalancer	One external IP (cloud-provisioned)	Production external access in the cloud
ExternalName	N/A — DNS alias, no proxying	Pointing at something <i>outside</i> the cluster
Ingress	One IP/port, routed by host/path	HTTP(S) routing to many Services

Mental model: ClusterIP is the foundation — every other type is built on top of it. NodePort = ClusterIP + a port opened on every node. LoadBalancer = NodePort + a cloud LB pointed at those node ports. Ingress = ClusterIP + a shared L7 router in front of it.

Lab Setup — create the working namespace

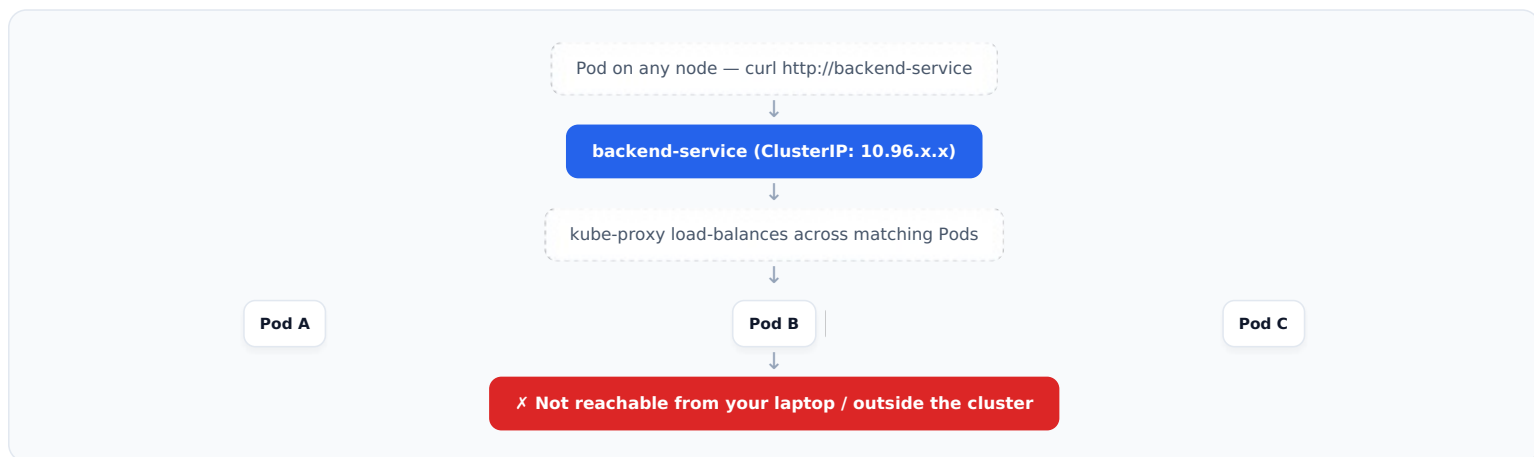
Everything in this lab lives in its own namespace so it's easy to clean up:

```
kubectl create namespace netlab
```

Keep this handy: to wipe the entire lab at any point and start fresh, run `kubectl delete namespace netlab` . All Deployments, Services, and NetworkPolicies inside it disappear with it.

Theory: The Default, Internal-Only Service

ClusterIP is the default Service type. It gets a stable virtual IP that only exists **inside the cluster's network** — kube-proxy programs routing rules on every node so that IP always lands on a healthy Pod, but nothing outside the cluster can reach it.



- ✓ Gets a virtual IP from the cluster's Service CIDR (e.g. `10.96.0.0/12`)
- ✓ Automatically gets a DNS name: `<service>.<namespace>.svc.cluster.local`
- ✓ Load-balances across every Pod matching its selector
- ✓ Is the building block every other Service type extends

03 Hands-on: ClusterIP

Task 1 — Deploy the backend

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: netlab
spec:
  replicas: 3
  selector:
    matchLabels: { app: backend }
  template:
    metadata:
      labels: { app: backend }
    spec:
      containers:
        - name: backend
          image: hashicorp/http-echo
          args: ["-text=Hello from backend", "-listen=:8080"]
          ports: [{ containerPort: 8080 }]
---
apiVersion: v1
kind: Service
metadata:
  name: backend-service
  namespace: netlab
spec:
  type: ClusterIP
  selector: { app: backend }
  ports: [{ port: 80, targetPort: 8080 }]
```

```
kubectl apply -f backend.yaml
kubectl get svc backend-service -n netlab
```

Task 2 — Prove it only works *inside* the cluster

Start a throwaway debug Pod and call the Service by its DNS name:

```
kubectl run curler --rm -it --image=curlimages/curl -n netlab -- \
  curl http://backend-service.netlab.svc.cluster.local
```

You should get back `Hello from backend`. Now try reaching it from **outside** the cluster:

```
curl http://<ClusterIP>:80 # from your laptop – this will just hang / time out
```

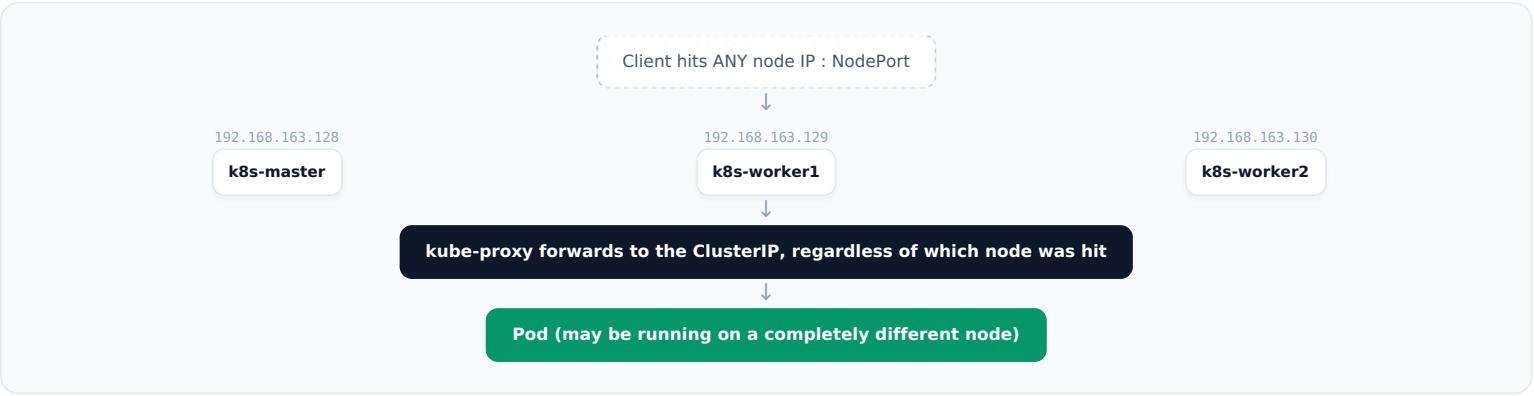
What to observe: the request from inside the cluster succeeds instantly; the request from your laptop never connects. Record the ClusterIP with `kubectl get svc -n netlab` and confirm it's in the `10.96.0.0/12` range (or whatever `--service-cidr` your cluster uses).

Checkpoint question

Why did `curl` from the debug Pod work by DNS name, but a Pod in a *different namespace* would need `backend-service.netlab.svc.cluster.local` instead of just `backend-service` ?

Theory: One Port, Every Node

NodePort builds directly on ClusterIP: Kubernetes still creates the ClusterIP, but *additionally* opens the same high port (default range **30000-32767**) on **every node in the cluster** — even nodes that aren't running the Pod. kube-proxy forwards the traffic internally to wherever a healthy Pod actually lives.



Key insight: it doesn't matter which node the Pod is actually scheduled on — *every* node listens on the NodePort and can forward you to the right place. That's what makes it usable without a load balancer, but it also means a client has to know at least one real node IP.

05 Hands-on: NodePort

Task 1 — Expose the backend as NodePort

```
apiVersion: v1
kind: Service
metadata:
  name: backend-nodeport
  namespace: netlab
spec:
  type: NodePort
  selector: { app: backend }
  ports:
    - port: 80
      targetPort: 8080
      nodePort: 30080
```

```
kubectl apply -f backend-nodeport.yaml
kubectl get svc backend-nodeport -n netlab
```

NAME	TYPE	CLUSTER-IP	PORT(S)
backend-nodeport	NodePort	10.96.x.x	80:30080/TCP

Task 2 — Hit all three node IPs from outside the cluster

```
curl http://192.168.163.128:30080 # master
curl http://192.168.163.129:30080 # worker1
curl http://192.168.163.130:30080 # worker2
```

NODE HIT	EXPECTED RESULT
192.168.163.128 (master)	200 OK — "Hello from backend"
192.168.163.129 (worker1)	200 OK — "Hello from backend"
192.168.163.130 (worker2)	200 OK — "Hello from backend"

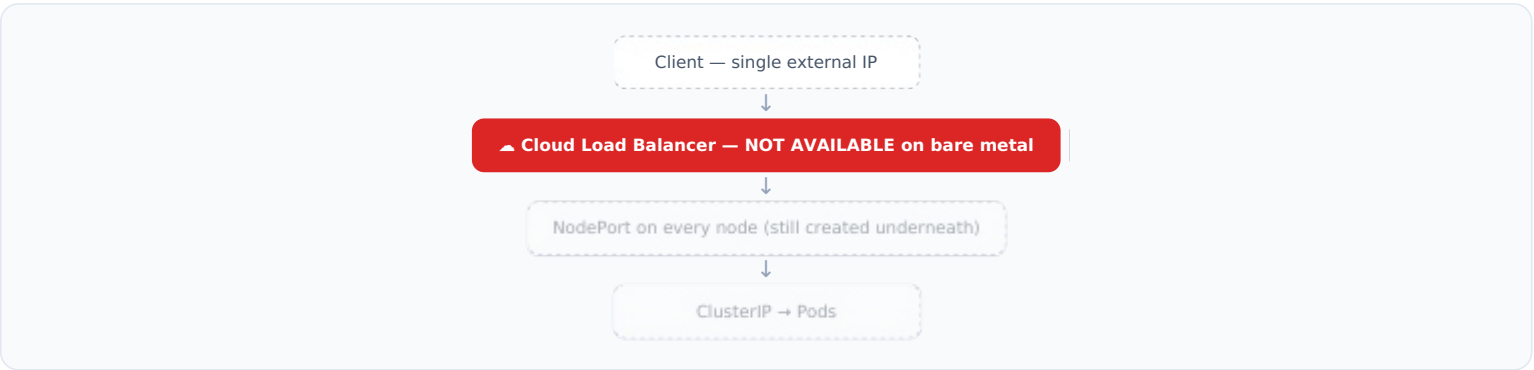
Task 3 — prove the routing, not just the port: run `kubectl get pods -n netlab -o wide` and note which node each `backend` Pod actually runs on. Then curl a node that has **none** of the backend Pods scheduled on it. It should still work — that's kube-proxy forwarding the request across the pod network to a node that does.

Real-world limitation: NodePort exposes a raw, unfriendly port (30080) on every node's IP, with no TLS termination, no host/path routing, and no single stable entry point. This is exactly the gap Ingress and LoadBalancer fill.

Theory: Cloud-Provisioned External IP

`LoadBalancer` is what you'd use in AWS, GCP, Azure, etc. Requesting this type tells the **cloud provider's controller** to provision a real external load balancer (an ELB, an Azure LB, a GCP forwarding rule...) and point it at the NodePort that Kubernetes opens automatically underneath.

✗ Your cluster (bare metal) No cloud provider integration No controller to provision an IP EXTERNAL-IP stays <pending> forever	✓ A cloud cluster (e.g. AWS/GKE) Cloud controller manager watches Services Provisions a real LB automatically EXTERNAL-IP fills in within ~1 minute
---	--



What we need here is a cloud Load Balancer. Your cluster has no cloud provider plugged in, so a `type: LoadBalancer` Service will create the NodePort layer just fine, but `EXTERNAL-IP` will sit at `<pending>` indefinitely — there's simply nothing watching for that request and provisioning an address.

The bare-metal alternative: MetalLB

On-prem clusters that still want a real `LoadBalancer` experience typically install **MetalLB**, which hands out IPs from a pool you own on your local network (e.g. `192.168.163.200–192.168.163.220`) and answers ARP/BGP for them. It's optional for this lab — we'll just observe the pending state, which is itself the important lesson.

07 Hands-on: LoadBalancer

Task 1 — Create a LoadBalancer Service

```
apiVersion: v1
kind: Service
metadata:
  name: backend-lb
  namespace: netlab
spec:
  type: LoadBalancer
  selector: { app: backend }
  ports: [{ port: 80, targetPort: 8080 }]
```

```
kubectl apply -f backend-lb.yaml
kubectl get svc backend-lb -n netlab --watch
```

Expected output — note `EXTERNAL-IP` :

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
backend-lb	LoadBalancer	10.96.x.x	<pending>	80:31xxx/TCP

Task 2: confirm the Service is still reachable via its auto-assigned NodePort (visible in the `PORT(S)` column), exactly like Part 3:

```
curl http://192.168.163.129:<auto-assigned-port>
```

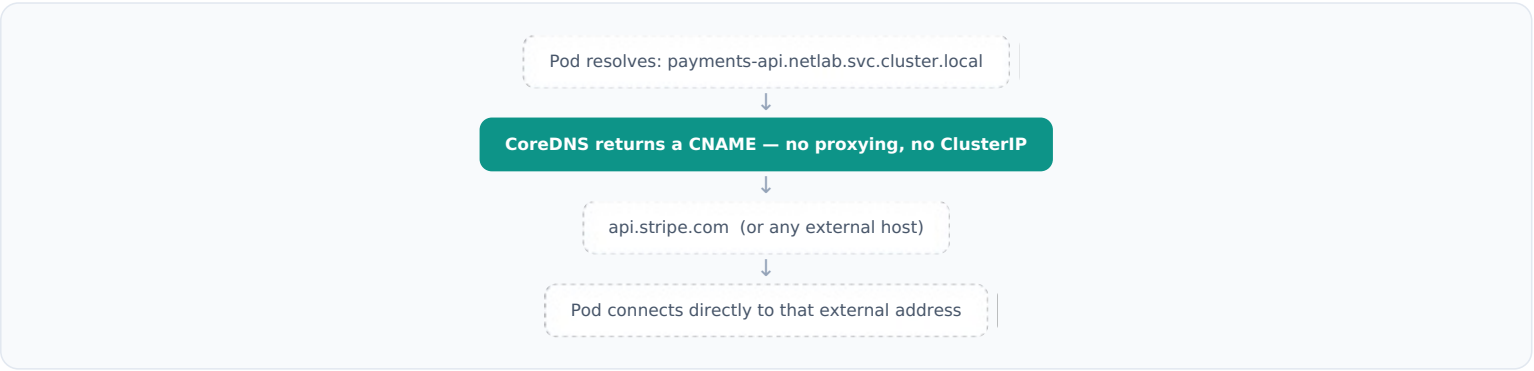
This proves `LoadBalancer` really is "NodePort + an external IP request" — the NodePort layer works even when the IP never arrives.

QUESTION	ANSWER TO FIND YOURSELF
Will <code>EXTERNAL-IP</code> ever change from <code><pending></code> ?	No — not without MetalLB or a cloud integration
Is the app still reachable?	Yes — via the NodePort, same as Part 3
What would fix this in production?	A cloud LB (AWS/GCP/Azure) or MetalLB on-prem

```
kubectl delete svc backend-lb -n netlab # clean up before moving on
```

Theory & Hands-on: The DNS-Only Service

`ExternalName` is the odd one out: it has **no selector, no ClusterIP, no kube-proxy rules at all**. It's purely a DNS CNAME record living inside the cluster's DNS, redirecting a Kubernetes-style name to an external hostname. Nothing is proxied — the client's own connection goes straight to the external address.



Task 1 — Create an ExternalName Service

```
apiVersion: v1
kind: Service
metadata:
  name: external-api
  namespace: netlab
spec:
  type: ExternalName
  externalName: example.com
```

```
kubecttl apply -f external-api.yaml
kubecttl get svc external-api -n netlab
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
external-api	ExternalName	<none>	example.com	<none>

Task 2 — Prove it's a DNS alias, not a proxy

```
kubecttl run dnstest --rm -it --image=busybox -n netlab -- \
nslookup external-api.netlab.svc.cluster.local
```

The answer should resolve straight through to `example.com`'s IP — there is no `10.96.x.x` ClusterIP anywhere in the chain.

When to use it: migrating a service from an external database to an in-cluster one (or vice-versa) without changing application config — the app always talks to `db.netlab.svc.cluster.local` ; only the Service definition changes underneath it.

Service Types — Full Comparison

TYPE	CLUSTERIP CREATED?	PORT ON EVERY NODE?	EXTERNAL IP?	ON YOUR CLUSTER
ClusterIP	Yes	No	No	Works as-is
NodePort	Yes	Yes (30000-32767)	No (use node IP)	Works as-is
LoadBalancer	Yes	Yes (auto)	Yes — cloud only	Stuck <pending> (needs MetalLB)
ExternalName	No	No	N/A (DNS alias)	Works as-is

Quiz 1

You need Pod-to-Pod traffic within the same cluster only. Which type, and why not NodePort?

Quiz 2

Your app needs to talk to a managed database outside the cluster, but you want app code to always use one internal-looking hostname. Which type?

Quiz 3

You deploy the exact same YAML on this cluster and on a fresh EKS cluster, using `type: LoadBalancer`. What's different about the result, and why?

Answers — (1) ClusterIP; NodePort would needlessly open an external-facing port for traffic that never leaves the cluster. (2) ExternalName. (3) On this cluster `EXTERNAL-IP` stays `<pending>` forever because there's no cloud controller manager; on EKS, AWS provisions a real ELB and fills the IP in automatically.

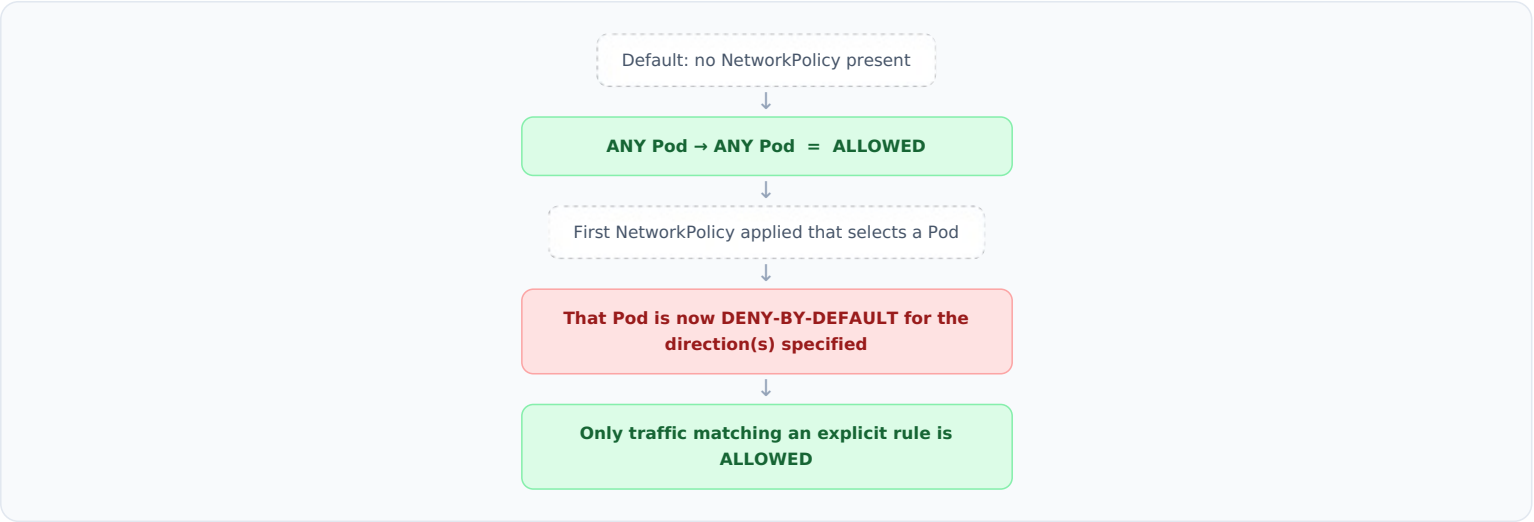
Theory: Kubernetes Is Open By Default

Here's the surprising part: by default, **every Pod can talk to every other Pod**, across every namespace, with zero restrictions. A `NetworkPolicy` is how you turn that open network into something with actual boundaries — a firewall that lives at the Pod level, selected by labels rather than IPs.

Before you start — check your CNI plugin. `NetworkPolicy` is a Kubernetes API object, but *enforcing* it is the job of your CNI plugin. Flannel (a very common default on kubeadm clusters) does **not** enforce `NetworkPolicies` — the objects will apply without error but traffic won't actually be blocked. Calico, Cilium, and Weave Net do enforce them.

```
kubectl get pods -n kube-system | grep -Ei "calico|cilium|weave|flannel"
```

If you see Flannel and nothing else, either install Calico in policy-only mode alongside it, or switch your CNI to Calico/Cilium before Part 7's tasks will actually take effect.



- ✓ `NetworkPolicies` are **additive** — multiple policies on the same Pod combine with OR logic
- ✓ A Pod with **no** policy selecting it stays fully open
- ✓ Policies select Pods with `podSelector`, and can restrict `ingress`, `egress`, or both
- ✓ Rules match by Pod label, namespace label, or IP block (`ipBlock`) — never by Pod name

11 Hands-on: Default Deny-All

Start by deploying a tiny 3-tier app to experiment on: **frontend**, **backend** (already deployed in Part 2), and **database**.

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: frontend, namespace: netlab }
spec:
  replicas: 1
  selector: { matchLabels: { app: frontend } }
  template:
    metadata: { labels: { app: frontend, tier: web } }
    spec:
      containers: [{ name: frontend, image: curlimages/curl, command: ["sleep","infinity"] }]
---
apiVersion: apps/v1
kind: Deployment
metadata: { name: database, namespace: netlab }
spec:
  replicas: 1
  selector: { matchLabels: { app: database } }
  template:
    metadata: { labels: { app: database, tier: data } }
    spec:
      containers: [{ name: database, image: hashicorp/http-echo, args: ["-text=DB OK","-listen=:8080"] }]
---
apiVersion: v1
kind: Service
metadata: { name: database-service, namespace: netlab }
spec:
  selector: { app: database }
  ports: [{ port: 80, targetPort: 8080 }]
```

```
kubectl apply -f 3tier.yaml
kubectl label pods -l app=backend tier=api -n netlab # tag backend too
```

Task — confirm everything can reach everything (the "before")

```
kubectl exec -n netlab deploy/frontend -- curl -s database-service
kubectl exec -n netlab deploy/frontend -- curl -s backend-service
```

Both should succeed — there's no policy yet, so the cluster is still fully open.

Now apply a default deny-all

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: netlab
spec:
  podSelector: {} # selects ALL pods in netlab
  policyTypes: ["Ingress", "Egress"]
```

```
kubectl apply -f default-deny-all.yaml
kubectl exec -n netlab deploy/frontend -- curl -s --max-time 5 database-service
```

Expected result: the `curl` now times out. An empty `podSelector: {}` matches every Pod in the namespace, and listing both `Ingress` and `Egress` with no rules underneath means **nothing in, nothing out** for any of them.

12 Hands-on: Allow Frontend → Backend Only

With deny-all in place, punch a precise hole: allow the **frontend** tier to reach the **backend** tier on port 8080, and nothing else.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
  namespace: netlab
spec:
  podSelector:
    matchLabels: { app: backend }
  policyTypes: ["Ingress"]
  ingress:
    - from:
      - podSelector:
          matchLabels: { app: frontend }
  ports:
    - protocol: TCP
      port: 8080
```

```
kubectl apply -f allow-frontend-to-backend.yaml
```

TEST	COMMAND	EXPECT
frontend → backend	<code>kubectl exec deploy/frontend -n netlab -- curl -s --max-time 5 backend-service</code>	✓ succeeds
frontend → database	<code>kubectl exec deploy/frontend -n netlab -- curl -s --max-time 5 database-service</code>	✗ times out
database → backend	<code>kubectl exec deploy/database -n netlab -- curl -s --max-time 5 backend-service</code>	✗ times out

Why frontend→database still fails: this policy only adds an *ingress* rule to **backend**. The `default-deny-all` policy from Task 1 still fully blocks **database** and still blocks **egress** from every Pod that doesn't have an explicit egress rule — including frontend's egress to anything but what you allow next.

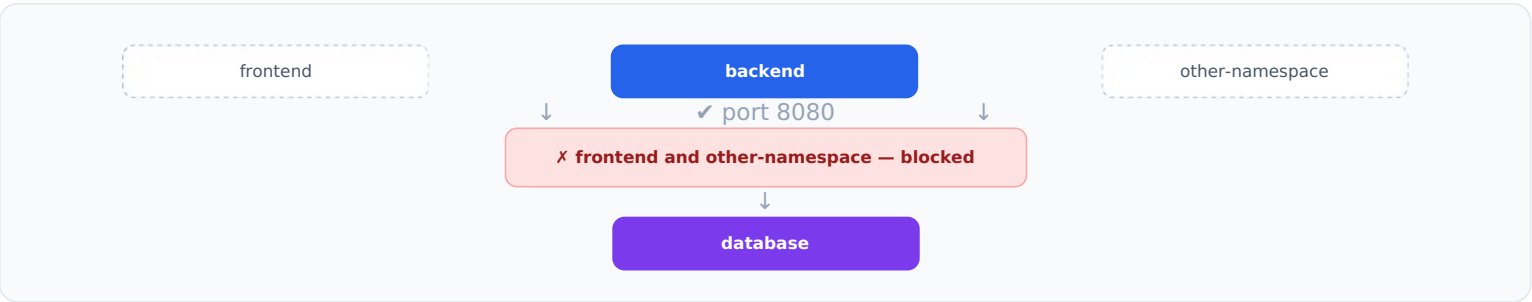
Extra task: frontend's *egress* is still fully denied by Task 1's policy too. Write a matching egress policy on **frontend** that allows egress to `app: backend` on port 8080 (and to `kube-dns` in `kube-system` on port 53/UDP, so DNS lookups keep working). Without the DNS rule, `curl backend-service` will fail at the hostname-resolution step even though the backend rule is correct.

13 Hands-on: Lock Down the Database Tier

The database should only ever hear from the backend — never from the frontend, and never from Pods in other namespaces. This is the pattern real production clusters use for the data tier.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-backend-to-database
  namespace: netlab
spec:
  podSelector:
    matchLabels: { app: database }
  policyTypes: ["Ingress"]
  ingress:
    - from:
      - podSelector:
          matchLabels: { app: backend }
```

```
kubectl apply -f allow-backend-to-database.yaml
```



Verification matrix — run all four and record the result

SOURCE	TARGET	EXPECTED
backend	database:8080	✔ Allowed
frontend	database:8080	✖ Blocked
frontend	backend:8080	✔ Allowed (Task 2)
a Pod in default ns	database:8080	✖ Blocked

Cross-namespace test: `kubectl run probe --rm -it --image=curlimages/curl -n default -- curl -s --max-time 5 database-service.netlab.svc.cluster.local` . Because the `podSelector` in `from` only matches label `app: backend` — regardless of namespace by default — add a `namespaceSelector` alongside it if you want to be extra explicit that only `netlab` 's backend counts.

Clean-up reminder: `kubectl delete networkpolicy --all -n netlab` removes all three policies if you want to reset connectivity to fully open before the final challenge.

Ingress Recap

From the previous lab: Ingress sits in front of ClusterIP Services and routes by **host** and **path**, using the NGINX Ingress Controller exposed via the NodePort you already set up (30080/30443). If it's not already installed on this cluster:

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/cloud/deploy.yaml
kubectl -n ingress-nginx edit svc ingress-nginx-controller # type: NodePort
```

Final Challenge — Full 3-Tier App

Combine everything from this lab into one working system. Use the `frontend` / `backend` / `database` Deployments already running in `netlab`.

- ☐ Expose **frontend** through an **Ingress** at host `netlab.company.local`, path `/`
- ☐ Expose **backend** through the *same* Ingress at path `/api` (ClusterIP underneath — no direct external access)
- ☐ Keep **database** on ClusterIP only, reachable exclusively from `backend` via NetworkPolicy
- ☐ Add an **ExternalName** Service called `partner-api` pointing at a mock external host, and confirm `backend` can resolve it
- ☐ Add your own hosts-file entry for `netlab.company.local` → `192.168.163.129` and test all three routes from your laptop browser
- ☐ Write the deny-all + two allow policies so the *only* permitted paths are: Ingress→frontend, frontend→backend, backend→database, backend→DNS/egress-to-partner-api

Success criteria: `http://netlab.company.local:30080/` and `/api` both work from your laptop; a direct `curl` to the database's ClusterIP from a debug Pod in another namespace fails; deleting the `allow-frontend-to-backend` policy immediately breaks the `/api` route without touching anything else.

Troubleshooting Cheat Sheet

SYMPTOM	LIKELY CAUSE	CHECK
ClusterIP not reachable even from inside	Selector/label mismatch	<code>kubectl describe svc <name> -n netlab</code> — check <code>Endpoints</code>
NodePort connects but hangs	Firewall blocking the NodePort range	<code>sudo ufw status</code> / cloud security group rules on all 3 nodes
LoadBalancer stuck <code><pending></code>	No cloud provider (expected here)	Use the auto-created NodePort, or install MetalLB
ExternalName doesn't resolve	CoreDNS not queried, or typo in hostname	<code>kubectl exec -- nslookup <svc>.netlab.svc.cluster.local</code>
NetworkPolicy has no effect at all	CNI doesn't enforce policies (Flannel)	<code>kubectl get pods -n kube-system</code> — look for Calico/Cilium
Policy blocks more than expected	Forgot DNS egress rule (port 53/UDP)	Add an explicit allow to <code>kube-system</code> / <code>kube-dns</code>
Ingress 404s on a valid path	Missing/incorrect <code>pathType</code> or backend port	<code>kubectl describe ingress <name> -n netlab</code>

Handy one-liners

```
kubectl get endpoints -n netlab           # confirm a Service has live Pods behind it
kubectl get networkpolicy -n netlab -o wide  # list all policies in the namespace
kubectl describe networkpolicy <name> -n netlab  # see exactly what a policy allows
kubectl exec -n netlab deploy/frontend -- nslookup backend-service
kubectl get svc -n netlab -o wide          # CLUSTER-IP / EXTERNAL-IP / PORT(S) at a glance
```

Full clean-up when you're done:

```
kubectl delete namespace netlab
```